Ejercicio 6

Polimorfismo vía Interfaces

Competencias por desarrollar:

- Diseña los componentes de un programa que resuelvan la situación planteada usando un diagrama de clases en UML.

- Implementa un programa que corresponda al diseño elaborado y resuelva la situación planteada usando los principios de diseño estudiados.
- Hace un buen uso de interfaces, aprovechando su flexibilidad.

Situación por resolver:

El backend de una plataforma de comercio electrónico debe interactuar con múltiples servicios externos: pasarelas de pago (tarjetas de crédito, transferencias bancarias, billeteras digitales), proveedores de logística para envíos y certificadores de Factura Electrónica en Línea (FEL). El equipo de arquitectura requiere un sistema central que gestione las comunicaciones con todas estas APIs de distintos proveedores, permitiendo integrar nuevos servicios externos en el futuro sin tener que reescribir el motor principal de la aplicación.

Algunas de estas APIs solo permiten consultas de estado o cotizaciones (como rastrear un paquete); otras ejecutan operaciones financieras de cobro y reintegro; y otras generan documentos legales (como los XML de las facturas o guías de envío). Para unificar la comunicación, el sistema estandarizará las interacciones mediante contratos como Transaccional, Documentable y Auditable. Si el sistema principal intenta procesar un reintegro financiero a través de la API de un proveedor logístico, el diseño debe abstraer esta limitación adecuadamente para ignorar la petición sin colapsar el proceso de compra ni lanzar excepciones en tiempo de ejecución.

Al iniciar, la aplicación ejecuta una carga inicial que construye el registro de integraciones con al menos 10 conexiones a APIs de naturaleza diversa. El administrador del sistema interactúa mediante un menú de consola que permite: listar todas las integraciones activas con su estado de conexión; buscar una API por identificador para revisar sus parámetros de configuración; y ordenar el catálogo por su tiempo de latencia promedio (milisegundos) para detectar cuellos de botella en la red. El equipo técnico ha garantizado que, cuando la empresa contrate a una nueva startup de entregas de última milla el próximo mes, bastará con instanciar la nueva integración en el catálogo para que funcione bajo los mismos contratos, sin alterar una sola línea de la lógica central de checkout.

Instrucciones

- Desarrolle un diagrama UML de su diseño. Asegúrese de aprovechar el uso de herencia y polimorfismo.
- No olvide emplear overloading donde lo necesite, ni de seguir los principios de diseño vistos en clase incluyendo la aplicación del patrón MVC.
- Implemente el prototipo incluyendo todas las funcionalidades requeridas. Su programa debe ser fiel a su diseño.
- El sistema debe tener solo una lista polimórfica que guarde todos los diversos ítems creados.
  Recuerde: haga primero su diseño. Si durante la implementación identifica la necesidad de cambios, detenga la programación y rediseñe antes de continuar. Documente los cambios con comentarios explicativos en el código.

Calificación

- **[50 pts.] Parte 1 -** Análisis y diseño.
  - **[10 pts.]** Uso correcto de una sola estructura polimórfica para almacenar los diversos objetos.
  - **[10 pts.]** Correcto uso de interfaces en las relaciones entre clases.
  - **[10 pts.]** Correcto uso de polimorfismo vía interfaces en la especificación de parámetros para comportamientos de los componentes y las especificaciones de *overloading*.
  - **[10 pts.]** Correcta separación de responsabilidades y aplicación de MVC y los principios de diseño para el cumplimiento de los requisitos funcionales.
  - **[05 pts.]** Buenas prácticas de programación:
    - *Override* de toString (e equals, donde sirva).
    - Encapsulación mediante modificadores de visibilidad y *getters/setters*.
  - **[05 pts.]** Correcta implementación de la interfaz Comparable para facilitar el ordenamiento de su lista polimórfica.

- **[50 pts.] Parte 2 -** Programa que implemente su diseño.
  - **[15 pts.]** Correcto uso de *polimorfismo*.
  - **[20 pts.]** Cumplimiento de requisitos funcionales.
  - **[10 pts.]** Usabilidad; interfaz amigable con el o los usuarios.
  - **[05 pts.]** Comentarios y encabezados.

Puntos Up (Hasta 20 puntos):

- (+20 puntos) Uso de interfaz gráfica.

Puntos Down (Hasta 100 puntos):

- (-30 puntos) Por cada While true y su correspondiente break.
- (-10 puntos) Por cada mensaje que muestre al usuario (System.out.println) que haga fuera de la clase Principal

Entregables

- Archivo .pdf con el análisis y diseño.
- Enlace al repositorio con el código que implementa las clases y el *driver program* requerido